

Introduction to Deep Learning

Conjugate Gradient Descent:-

* Instead of adjusting the η (learning rate) to accelerate the gradient descent, we can change the gradient computation itself.

* ~~Instead of~~ In the steepest Gradient Descent, in every iteration, the 90° direction shift happens from the previous gradient direction on the projection surface.

* In CGD, the 90° orthogonal direction shift happens on the loss surface directly. Because of which the minimum obtained in one direction will not be overwritten.

* At step n , d_n is the direction along which we are descending,

→ Avoid, $d_n^\top d_{n-1} = 0$, by setting $d_n = g_n + \beta d_{n-1}$

such that they are conjugate directions:-

$$d_n^\top H d_{n-1} = 0$$

(the 90° direction shift on the loss surface is ensured by this formula).

* Computing H is computationally expensive, so the following approximation is used.

Fletcher-Reeves approximation:-

$$\beta = \frac{-g_n^T H d_{n-1}}{d_{n-1}^T H d_{n-1}} = \frac{-g_n^T g_{n-1}}{g_{n-1}^T g_{n-1}}$$

* Update step,

$$w_n = w_{n-1} - \eta d_n$$

Newton's Method:- (2nd order):-

$$* L(w + \epsilon) = L(w) + \nabla L(w)^T (w + \epsilon - w) + \epsilon^T H \epsilon / 2$$

$$* L(w + \epsilon) = L(w) + \nabla L(w)^T \epsilon \rightarrow \text{1st order Taylor series expansion}$$

$$\nabla L(w + \epsilon) = \nabla L(w) + H \epsilon \rightarrow \text{2nd order Taylor series expansion.}$$

* $H \epsilon$ - is the change of gradient along the direction ϵ .

* $\boxed{V^T H \epsilon = 0}$:- V is the direction orthogonal to the change of gradient.

* If the gradient at the next step has to be zero, then,

$$\boxed{-\epsilon \approx H^{-1} \nabla L(w)}$$

To prevent instability,

$$-\epsilon \approx (H + cI)^{-1} \nabla L(w)$$

* Update step,

$$\boxed{w_n = w_{n-1} - \eta (H + cI)^{-1} \nabla L(w_{n-1})}$$

* Convergence :- $O(\gamma_n^n)$ - superlinear
- computationally ultra expensive.

Brayden - Fletcher - Goldfarb - Shanno Optimizer (BFGS) :-

* Computing Hessian for the loss function is so extremely expensive and will be highly impractical.

* So, we will approximate it with this BFGS optimizer.

$$f(w) =$$

$$L(w+\epsilon) \approx L(w) + \nabla L(w)^T \epsilon$$

$$\nabla L(w+\epsilon) \approx \nabla L(w) + H \epsilon$$

$$g_n \approx g_{n-1} + H(\omega_n - \omega_{n-1})$$

$$\Rightarrow g_n - g_{n-1} \approx H(\omega_n - \omega_{n-1})$$

$$\Rightarrow h_n \approx H(\omega_n - \omega_{n-1})$$

$$\Rightarrow h_n \approx H d_n$$

Hessian at that location,

$$H_n d_n \approx h_n$$

$$\begin{cases} \text{change in} \\ \text{weights,} \\ d_n = \omega_n - \omega_{n-1} \\ \text{change in} \\ \text{gradients,} \\ g_n = h_n = g_n - g_{n-1} \end{cases}$$

This is of the form, $Ax = B$

from this, $A = Bx^{-1}$

This turns out to be an optimization problem,

we will find out a matrix A , such that it is symmetric ($A = A^T$) and $Ad_n = h_n$ condition gets satisfied and it is close to H_{n-1} ($\arg\min \|A - H_{n-1}\|$).

Solve, $\arg\min \|A - H_{n-1}\|$ such that, $A = A^T$ and $Ad_n = h_n$

The solution to this optimization problem becomes,

$$H_n^{-1} \approx B_n = B_{n-1} + \left(d_{n-1}^T d_{n-1} / h_{n-1}^T d_{n-1} \right) - \left(B_{n-1} h_{n-1} h_{n-1}^T B_{n-1} / h_{n-1}^T B_{n-1} h_{n-1} \right)$$

Initialize:- $B_n = 0$; ($H_0 = 0$);

↓
we are directly getting H^{-1} with relatively lesser computation (all these are vector inner products).

Update step:-

$$w_n = w_{n-1} - \eta B_{n-1} \nabla L(w_{n-1})$$

This is computationally expensive than 1st order techniques, but it converges much faster.

L-BFGS:-

Instead of storing all the columns of the Hessian matrix (the complete matrix), the L-BFGS stores only the important columns of the Hessian matrix every iteration, this way is still computationally expensive, but converges faster.

Nesterov Optimization (N-Adam):-

- * Evaluate gradient / Hessian near the update point.
- Predict the next point and find the gradient there (gradient look ahead).

* Compute, $\nabla_{\omega_{n-1} + \mu \tau_{n-1}} L_i$ instead of $\nabla_{\omega_{n-1}} L_i$

$\rightarrow \gamma_n$ is the parameter update term such that

$$\omega_n = \omega_{n-1} + \gamma_n$$

* NAdam has the faster convergence rate in term among the 1st order methods:- $O(1/n^2)$

All these techniques are invented for the noisy loss function optimization. Is there a way to make the loss function less noisy.

Regularization:-

* Improve weight/parameter selection through prior information, (by adding constraints on the parameters).

* The new loss function becomes,

$$L(\omega)_{\text{new}} = L(\omega) + \frac{\lambda}{2} \|\omega\|^2$$

The smallest value, it can take is 0, this is called weight decay.

$\rightarrow$ Weight decay.

$$\omega \sim \mathcal{N}(0, \sigma^2)$$

The gradient of the ^{new} loss function is,

$$\nabla L_{\text{new}} = \nabla L + \lambda \omega$$

The weight update is,

$$\omega_n = \omega_{n-1} - \eta (\nabla L + \lambda \omega_{n-1})$$

$$\omega_n = (1 - \eta \lambda) \omega_{n-1} - \eta g_{n-1}$$

This is the Ridge Regression.

* This coupled with NAdam is called NAdamW.

The W - indicates weight decay.

* L1 regularization is used for sparse weights/parameters, this pushes the weights exactly to zero.

* Applying L1 (Lasso) regularization is equivalent to pruning in neural network.

* Parameter sharing :- Constrain the weights to be similar to another fixed pretrained neural network:

$$L(\omega) + \lambda \|\omega - \omega_{\text{fixed}}\|^2$$

How to improve Generalization performance?

and Avoid overfitting?

Data Augmentation:-

- * Increasing training data improves loss surface & generalization.
- * Noise injection / perturbation :- Add noise to training data
- * Adding Gaussian noise to input has the same behavior as L2 Regularization.

$$L(w) = |y - w^T(x + \epsilon)|^2 \approx |y - w^T x|^2 + \lambda \|w\|^2$$

- * Computing gradients with the augmented data will be computationally expensive than applying L2 Regularization. So, L2 Regularization is generally preferred.

- * Label Smoothing :- Change 0/1 labels as ϵ and $1-\epsilon$ (label noise).

In case of a softmax function, so ~~also~~ to reduce the loss, $L(w) = H(y; \sigma_k(z)) = \sum_{i=1}^K y_i \log \sigma_i(z)$; the ' y ' should be 0, for ' y ' to be 0, e^z should be 0 and ' z ' should take $-\infty$, for ' z ' to take $-\infty$, $w^T x + b \rightarrow -\infty$, and ' w ' should be $-\infty$, this is a problem, we cannot perform gradient descent on ' w '.

To overcome this, we do Label smoothing, $\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \rightarrow \begin{bmatrix} \epsilon/k \\ 1-\epsilon/k \\ \epsilon/k \\ \epsilon/k \end{bmatrix}$

This is created by the convex combination of an uniform distribution and one-hot encoding. $z = (1-\epsilon)y + (\epsilon)u$

$$\tilde{y} = (1-\epsilon)y + \epsilon u = \left[\frac{\epsilon}{k}, 1-\epsilon+\frac{\epsilon}{k}, \frac{\epsilon}{k}, \frac{\epsilon}{k} \right]^T$$

$$\tilde{L}(w) = (1-\epsilon) \cdot L(w) + \epsilon \cdot H(u; \sigma_k)$$

- * Bootstrapping using auto encoders.
 - * Learn data distribution and sample from data distribution.
- Unsupervised Learning
(Semi-supervised learning)

Bagging / Ensemble Learning :-

Bootstrapping:- Create multiple training datasets by drawing a subset of samples from the original dataset with replacement. Multiple models will be developed with these datasets.

Ensemble:- Multiple models (equivalent to the no. of datasets) will be developed and aggregation is done.

→ Classifier:- Maximum voting based inference

→ Regression:- Average all the outputs.

Bagging:- Bootstrap aggregation.

* If the error of each network is e_m , then the

bagging error is $\sum e_m / M$

* The errors are correlated.

$$\left\{ \begin{aligned} e_m(x) &= E \{ f_m(x) - y(x) \} \\ E \{ (f_m(x) - y(x))^2 \} &= \\ E \{ \left(\sum (e_m(x)) / M \right)^2 \} \end{aligned} \right.$$

Teacher Training and Knowledge Distillation:

- * Train a smaller (student) network ~~from~~ using a larger (teacher) network.
- * Knowledge from deep teacher net is distilled to a shallow student net.
- * Teacher net is a ^{conventional} deep net with CE loss.
- * Student's loss:
$$(1-\epsilon) H(\text{input}; \text{label}) + \epsilon H(\text{input}; \text{teacher's output})$$
- * Teacher's outputs are soft labels - smoothen;
 ϵ depends on temperature parameter.
- * Another possible loss:
$$\text{MSE}(\text{input}; \text{teacher's logits})$$
- * Better generalization. $\hookrightarrow$ minimize the loss b/w student's output & teacher's output first.

Semi-Supervised Learning:-

- * Similar to data ~~distribution~~ augmentation - generate new data samples instead of adding noise to existing samples.
- * Unsupervised Learning: Learn data distribution - joint PDF $P(\text{input}, \text{label})$ and sample from this distribution to get new samples.
- * Autoencoders:- A non-linear PCA like unsupervised learning

→ Train an autoencoder with given dataset, it reduces the dimension and the decoder is fed with some noise to generate new data.

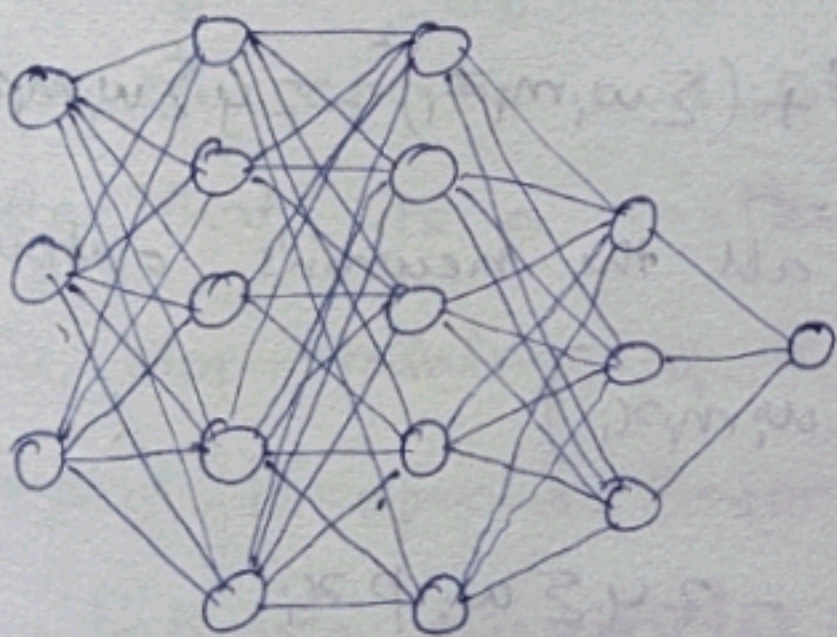
Dropout:-

Can we train multiple neural networks with just one neural network? → Dropout!!!

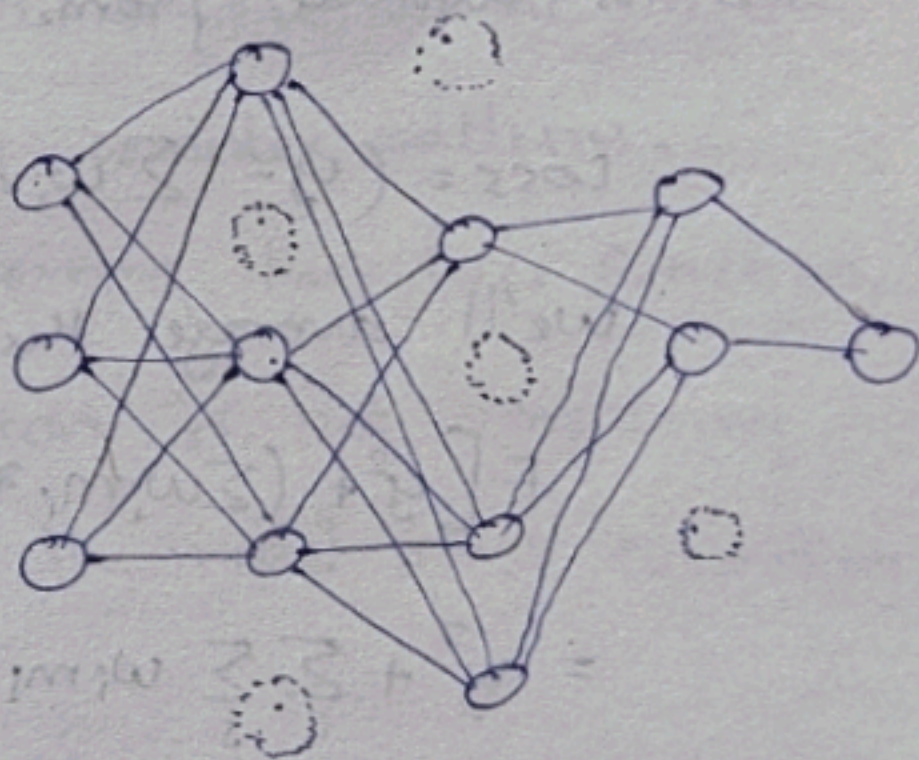
→ Another way to implement bagging without high computational complexity.

- * Randomly drop a fraction (p) neurons in hidden layers during training. (in each iteration, diff. neurons get dropped).
- * Dropout (regularization) - practically effective

→ Multiplicative noise on weights.



Original Neural Network.



Drop out in one iteration.

- * Because we are dropping different neurons randomly in each iteration, we are effectively training ^{multiple} different models.

* At the end of training, we might have applied different masks (to different neurons). so, scale the output from each and every edge with $\boxed{p}$, with which we are averaging the entire set of neural networks learned so far.

$$m = \begin{cases} 1 & ; p \\ 0 & ; 1-p \end{cases}$$

(mask)

$$E[m] = p$$

→ multiplied by p.
→ equivalent to

→ averaging.

$$E[m_i] = E[m_i^2] = p; E[m_i m_j] = p$$

* Dropout method approximates training an exponential no. of sub-models simultaneously.

$$\text{Loss} = (y - p w^T x)^2 + p(1-p) \sum w_i^2 x_i^2$$

This is derived from, (TLDR;)

$$\text{Loss} = (y - \sum w_i m_i x_i)^2 = y^2 + (\sum w_i m_i x_i)^2 - 2y \sum w_i m_i x_i$$

we'll average the loss of all the neuron's output

$$E \left[y^2 + (\sum w_i m_i x_i)^2 - 2y \sum w_i m_i x_i \right]$$

$$= y^2 + \sum_i \sum_j w_i m_i x_i w_j m_j x_j - 2y \sum w_i p x_i$$

$$= y^2 + \sum_{i=j} w_i^2 m_i^2 x_i^2 + \sum_{i \neq j} \underbrace{w_i w_j m_i m_j x_i x_j}_{p^2} - 2y \sum w_i p x_i$$

$$= (y - p w^T x)^2 - \underbrace{p^2 (w^T x)^2}_{(1-p)} = y^2 - 2y p w^T x$$

If $E[x_i^2] = \text{variance} = 1$, then loss

$$\text{loss} \approx (y - pw^T x)^2 + p(1-p) \|w\|^2$$

This is same as regularization.

Note:-

To add noise to the inputs, "add it" $\rightarrow$ Noise injection

To add noise to the labels, "take convex combination" $\rightarrow$ Label Smoothing

To add noise to the weights, "multiply them - with prob. p " $\rightarrow$ Dropout.

$\Rightarrow$ All are some way or the other,
equivalent to Regularization.

Early Stopping:-

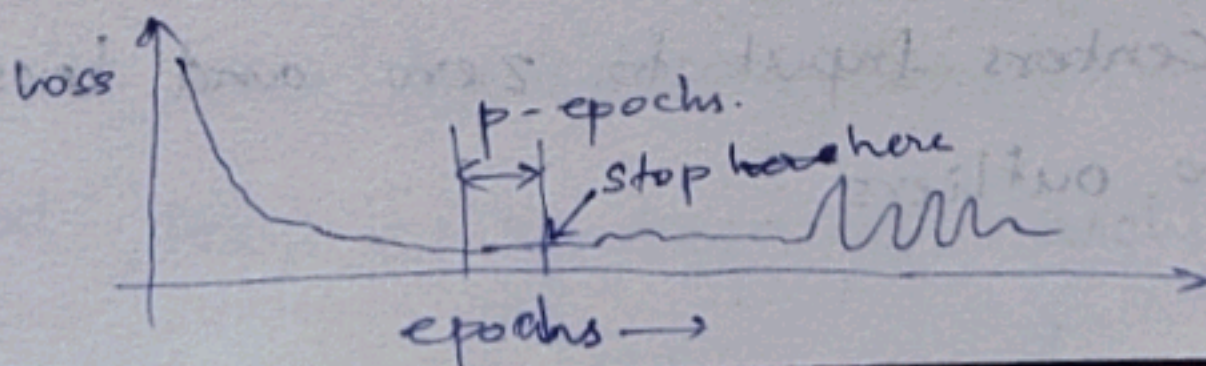
* During training: The simple patterns are learnt in the first few epochs and followed by complex patterns.

After that, its a matter of converging the loss function.

* So, stop training, if the validation is not improving beyond ' p ' epochs.. Beyond which the model start overfitting.

Patience parameter:- $\boxed{p}$

* Produces similar effect as L2 Regularization.



- * Early stopping is not good for extremely large models as it affects Generalization.
- * Typically, early stopping is applied when the data is noisy.

Exploding / Vanishing gradients:-

All of the regularization methods, by some means, attempts to keep the gradients stable.

Input Normalization:-

Scale input features to prevent gradient vanishing/exploding. Preserves input distribution shape but not scale or shift.

$$\text{Min-Max scaling: } \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- * Scale data to $[0, 1]$ (or) $[-1, 1]$
- * Sensitive to outliers and corrupted data

$$\text{Z-score standardization: } \frac{X - X_{\text{mean}}}{\sqrt{X_{\text{var}}}}$$

- * Centers Input to zero and less sensitive to outliers.

Distribution shift across layers:-

$x_2 = f_1(W_1 x_1 + b_1)$:- When the weights change during learning, the distribution of x_2 changes as well. This is called Internal Covariate Shift.

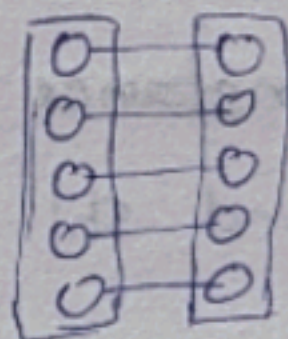
So, we need to keep normalizing at each layer.

Batch Normalization:-

Normalizing the data / logits b/w layers. The normalizing happens by taking mean and variance with the logits. In batch normalization, the mean and variance are calculated for all the outputs from a single neuron, and ultimately for each neuron, the same way.

$$\mu = \sum x_i / m \quad ; \quad \sigma^2 = \sum (x_i - \mu)^2 / m$$

$$y_i = \beta + \gamma (x_i - \mu) / (\sigma + \epsilon)$$



→ β and γ are learned parameters.

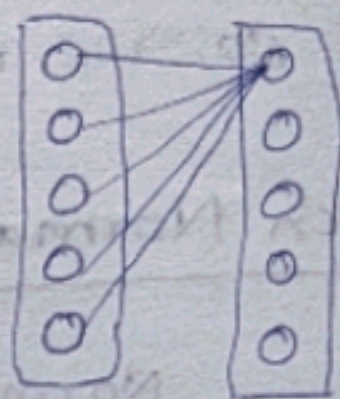
- * Behaves like noise injection, and hence, regularization.
- * Larger batch sizes provide better estimate of statistics.
- * It is implemented as a separate layer by itself along with its own backpropagation.
- * Works good for CNN type of architectures.

Layer Normalization :-

- * The concept remains the same as Batch normalization
- * The difference is, the mean and variance are calculated across neurons in a layer. i.e., i^{th} output of all neurons.

$$\mu = \sum x_i / d ; \quad \sigma^2 = \sum (x_i - \mu)^2 / d$$

$$y_i = \beta + \gamma (x_i - \mu) / (\sigma + \epsilon)$$



- * When there are multiple channels, all of them are included in Layer Normalization.
- * Observed to be good for RNN type of architectures.

Other Normalization techniques :-

Instance Normalization :- When there are multiple channels and still layer norm is applied to only one channel.

Group Normalization :- When there are multiple channels and layer norm is applied to a subset of channels.

Note :- Batch normalization is preferred when the neurons in a layer are uncorrelated and the viceversa for Layer Normalization.

Curriculum Training:-

Can the shift of variance be handled by varying the dataset used for training?

→ change the distribution of data over epochs.

* Train the NN from easier to harder, to learn the data.

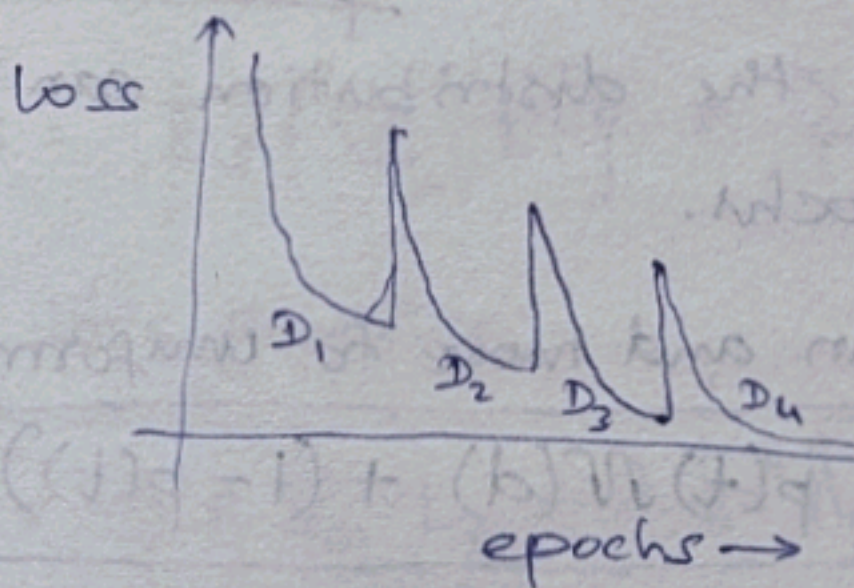
* Curriculum:- A sequence of training dataset $- D_1, D_2, \dots, D_T$

* Separate the datasets based on domain knowledge.

* Curriculum training is same as regularization.

* Keep some portions of the previous datasets to avoid forgetting the learnings got earlier.

* Causing distribution shift to learn - loss surface changes with distribution.



→ How do we say if a datapoint is easy or difficult to learn? How do we separate the dataset?

Scoring and Pacing functions :-

* How to measure difficulty: Scoring function $S(d)$.

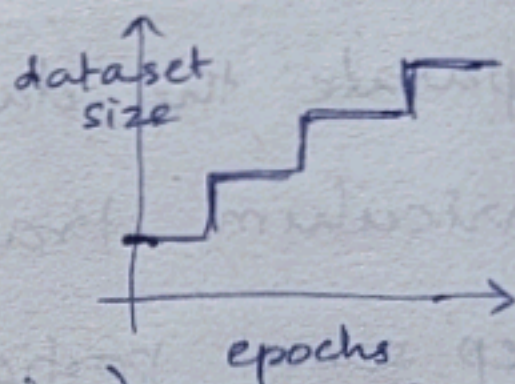
→ No. of samples, No. of classes, No. of samples per class, entropy, SNR/distortion, distance/norm of feature vectors/embedding, sentence/sequence rarity, etc:-

* When to train with a difficult sample: pacing function $p(t)$

» Discrete:-

change dataset at fixed intervals

- $D_1 \ll D_2 \ll D_3 \ll \dots \ll D_T$
(or subset from previous)



» Continuous:-

change dataset at each epoch

- Sample from a distribution
- The parameters of the distribution can be updated over epochs.
- start with Gaussian and move to uniform:



$$f(t, d) = p(t)N(d) + (1 - p(t))U(d)$$

- start with a low variance in Gaussian distribution and slowly increase the variance until uniform distribution is attained.

- The normal distribution with very low variance picks the dataset with score close to 0, upon

moving and at uniform distribution, the scores 0 and 10^6 will also be there.

- The rate at which we are moving towards harder dataset is defined by the pacing function.

All these are good, if we know how to define the scoring and pacing function. Otherwise, the Self-Paced Learning is preferred.

Self-Paced Learning:-

If we don't know how to define our scoring and pacing functions, then we'll redefine the loss function in a way that it gets adjusted to learn from the datapoints that have lower loss and gradually moves to higher loss, datapoints.

$$\text{Loss} = \mathbb{E}_f \left\{ \nabla_{w_{n+1}} L_i \right\} = \sum_i \nabla_{w_{n+1}} L_i / N$$

for uniform distribution
(considering all the data points irrespective of difficulty levels).

Redefined loss function:-

$$\text{Loss} = \sum_i f_i \nabla_{w_{n+1}} L_i - \lambda \|f\|_1$$

Here, $f_i \rightarrow 0$ (or) 1 for a datapoint.

- * Samples with lower loss gets selected first.
- * Harder samples have high loss values - higher variance of gradients.
- * This method reduces gradient variance early in training.

Eg:- We have two data points.

$D_1 \rightarrow$ with loss value $L_1 = 0.1$

$D_2 \rightarrow$ with loss value $L_2 = 3$

$$\text{loss} = f_1 L_1 + f_2 L_2 = f_1 \times 0.1 + f_2 \times 3$$

if we choose, $f_1 = 0, f_2 = 1 \rightarrow \text{Loss} = 3$

$f_1 = 1, f_2 = 0 \rightarrow \text{loss} = 0.1$

$f_1 = 1, f_2 = 1 \rightarrow \text{loss} = 3.1$

* Otherwise, Reinforcement learning can be used to dynamically change dataset.

* Often used in NLP & CV, wherever dataset have high variance.

Transfer Learning:-

What if we initialize a neural network, or parts of it, with weights / parameter from another pre-trained model?

$\rightarrow$ This can be seen as a special case of curriculum learning or teacher training.

Two goals:- Parameter transfer (or) Feature representation transfer.

Inductive (different task) v/s Transductive (same task diff dataset)

NN output:-

$$y(w, x) = g(w_1, h(w_2, x))$$

$h() \rightarrow$ Feature extractor

$g() \rightarrow$ Task specific inference function

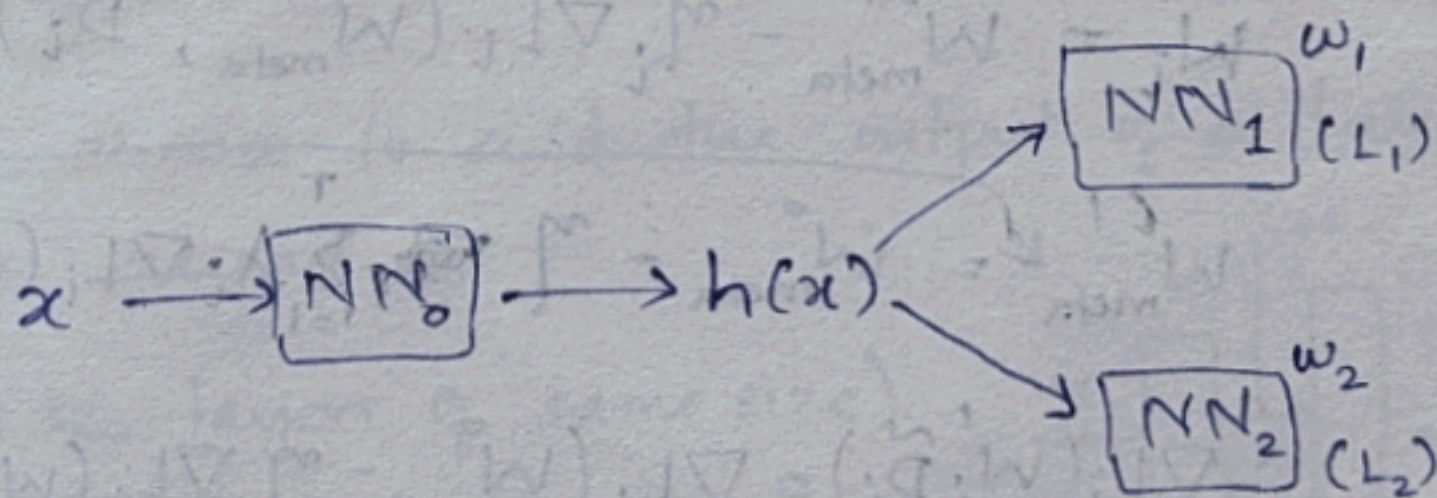
Here the $g()$, gets the weights w_1 learned by $h()$ and starts learning w_2 .

* When we do transfer learning, we may want our model to not drift off very far away from the pre-trained (current) model. To prevent this model drift, the regularization is added.

$$L(w) = + \lambda \|w - w_{\text{pretrained}}\|^2$$

$\rightarrow$ This is primarily used in transductive learning.

What if we want to train two distinct models for two different tasks, both sharing a common feature extractor?



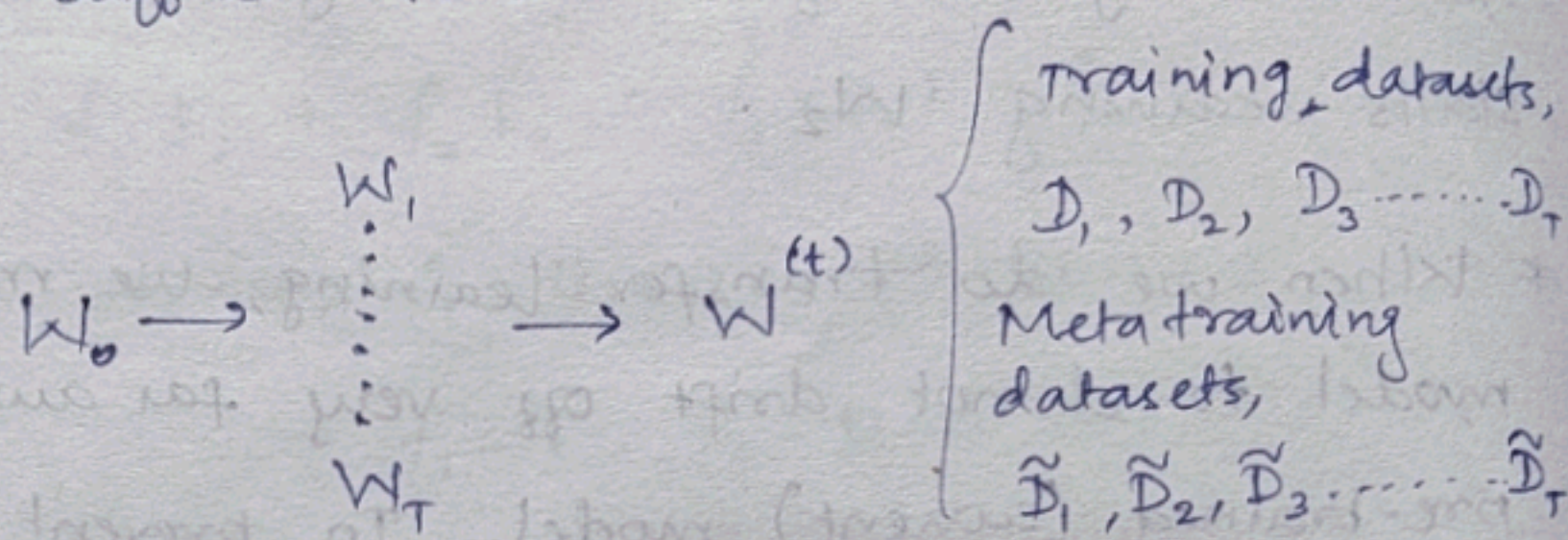
This is called Multi-task learning and the loss function becomes,

$$\text{loss: } \min_w L_1(w_1, h(w_0, x)) + \lambda L_2(w_2, h(w_0, x))$$

Now the training of the whole network can be done together with this loss function.

Meta Learning & Few-shot Learning

How to learn different model parameters for different tasks (loss) with different train and test (correlated) datasets?



(outer loop)

Meta Learning :- $\min_i \sum L_i(W_i, D_i, \text{meta learning training})$

(inner loop)

task specific learning :- $W_i = U(W, D_i, \text{training})$

For NIN i ,

initializing W_i as the meta weight

$$W_i^0 = W_{\text{meta}}^0$$

$$W_i^1 = W_{\text{meta}}^0 - \eta_i \nabla L_i(W_{\text{meta}}^0, D_i)$$

This meta weight is learnt from all the training datasets used for task specific learning, so that W_{meta} can be common for all the sub tasks.

$$W_{\text{meta}}^1 = W_{\text{meta}}^0 - \eta \sum_{i=1}^T \lambda_i \nabla L_i(W_i^1, \tilde{D}_i)$$

$$\nabla L_i(W_i^1, \tilde{D}_i) = \nabla L_i(W_{\text{meta}}^0 - \eta_i \nabla L_i(W_{\text{meta}}^0, D_i), \tilde{D}_i)$$

Hessian

So, meta learning is computationally very expensive.

If we cannot avoid meta learning, then we can prefer

First Order approximation of Meta Learning. This is called Mod

Intuitiveness:-

The weights of task specific NN are all random variables, ^{which are correlated.} and the neural networks share some common information. So, there exists a prior to these random variables, and generate these weights from the prior.

This is similar to a Bayesian model of learning a prior. This Bayesian prior will capture all set of ~~meta~~ meta weights which can be used to come up with optimal weights suitable for such specific tasks.

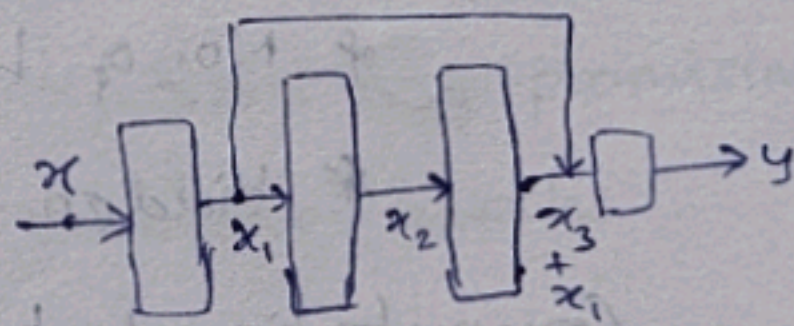
Few-Shot Learning:-

Once the meta weights are learnt, fine tuning the weights of the task specific networks is called Few-Shot Learning. D_i is the small dataset.

Skip Connections:-

Randomly choose to add the output of a layer to that of another layer ahead.

- short skip (b/w layers of same size)
- Long skip (brings in features that may have been 'forgotten').



- Has a behavior similar to ensemble learning.

Eg:-
$$f_3 \left(W_3 \left\{ f_2 \left(W_2 f_1 \left\{ W_1 x + b_1 \right\} + b_2 \right\} + f_1 \left(W_1 x + b_1 \right) \right\} + b_3 \right)$$

$\underbrace{\hspace{10em}}_{x_3} \qquad \qquad \qquad \underbrace{\hspace{10em}}_{x_1}$

* Skip connections via concatenation.

→ Increase output size - Dense Net.

* Skip connections are useful in avoiding vanishing gradients primarily, as the concatenated term becomes 1 during gradient calculation which gets added to the gradient term if it is very small, the concatenated term revives it.

Hyper parameter tuning:-

Typical Hyperparameters:-

* Learning rate

* Momentum factors

* Weight decay regularization coefficient

* Drop out rate

* Label smoothing Level

* Training batch size

* No. of Layers

* Hidden units, etc.:-

Computational tools used:-

* Hyper opt

* Skopt

* Ray tune

* Wandb

* Optuna

How to find the optimal hyperparameters?

$$\rightarrow \underset{\lambda}{\operatorname{argmin}} \min_w L(w, \lambda)$$

$\rightarrow$ Gradient descent over λ ? (GD & GD - Hessian! computationally expensive)

* Train model with different hyperparameter values and pick the one with the least validation error.

* **Grid Search**:- Search from a pre-fixed set of values.

* **Random Search**:- Sample from a distribution

$\rightarrow$ Learning rates - log uniform search

$\rightarrow$ drop outs - uniform search

$\rightarrow$ Bayesian search: fit a distribution to the p hyperparameters, estimate its parameters and sample from it.

* **Hyperband**:- Pick multiple candidates, train shortly

(early stop), drop high-loss ones and keep only promising ones - dynamically allot more/less training epochs.